What does the model think it knows? A latent world model of LLM belief updates

Javier Fumanal-Idocin

October 29, 2026

Abstract

When an LLM solves a problem it is holding a belief of the state that is updated as new evidence arrives. Prior mechanistic interpretability literature has used linear probes to understand where and how this belief state "lives" inside the model and how it affects its decisions. However, interacting with this belief state or understanding its internal dynamics is still problematic. In this work, first we locate the belief state in two games, a clue game and a text dungeon, where it is well defined and can be intervened, and we compare it with a Bayes-optimal model. Secondly, we use a Joint Embedding Predictive Architecture (JEPA) to learn the transition between different moments in the game, and to understand the "world model", or lack of it, of the LLM. Based on the JEPA latent space, we are able to understand the effect of each clue and why the LLM shows significant differences with the Bayes optimal solutions. For our experiments, we use Qwen2.5-1.5B-Instruct and Qwen3.5-2B. Our results show (...) PENDING.

1 Introduction

As Large Language Models (LLMs) are deployed in commercial products, there has been a rising awareness of their problematic behaviour in some cases. LLMs can help malicious users develop weapons, perform cyberattacks or develop fraud at scale by doing more sophisticated forms of spam [20]. Beyond malicious users, LLMs can also tell a user to hurt themselves or use inappropriate language to answer the user. Because of that, multiple research lines have sprung up regarding AI safety. One of the most popular ones is preference learning, in which the model is reinforced to act in a particular way, rejecting prompts that can result in undesired behaviour [14, 31]. Another popular approach is mechanistic interpretability, in which we aim to understand the computation of the model in terms of circuits [30, 17, 44], which can then be intervened to mitigate inappropriate behaviour [40, 49]. There are also other complementary techniques, such as constitution-based models, which use conditioning prompts to align the model with the desired behaviour [6]. Finally, it is also possible to look for safety-by-construction methods, in which the resulting LLM is safe to use based on some architectural constraint [15].

However, most approaches fail to completely delete undesired behaviour, and even though frontier models are more capable of resisting jailbreaks, these are still possible [45, 50]. Moreover, they still show tendency to deceive and other undesired traits when stress tested for this in modern benchmarks [34, 28], which shows that we are still far from having completely reliable AI models. Part of this unreliability comes from the model not understanding the problem it is in: a model can learn the wrong goal from a correct specification and pursue it competently [24, 36], its implicit model of a task can be incoherent even when its predictions are good [43], and whether it reports what it internally believes is the question behind honesty and eliciting latent knowledge [18, 13]. All of these are statements about the belief state of the model, which is what we try to trace here.

In this work, we aim to obtain with a post-hoc model, the insight about how the LLM understands a problem that the LLM architecture itself cannot give us. Our proposal is to use a Joint Embedding Predictive Architecture (JEPA) [25, 4] to model the belief transitions of an LLM while solving a particular problem. JEPAs use a latent space that incentivises the model to understand the problem it is solving. This has been

referred to in the literature as "having a world model" [25, 19], in the sense that the JEPA needs to understand the dynamics of the phenomenon instead of merely learning, for example, pixel or word correlations.

There have been some criticisms about LLMs lacking such a world model [10, 43], which can make them structurally unreliable. In this work, we propose to use a JEPA model to learn a model of the LLM's belief dynamics, that is, what the LLM is really processing in each step of a problem resolution. For that, we first look where to find the "belief state" of the model (in which neural activations does the model reflect its belief about the problem state). This has been regularly done in mechanistic interpretability research with linear probes [2, 26, 37, 48], although we found that for specific problems and depending on the kind of reasoning required, this can require different layers per problem. Then, we use that to train a JEPA model that learns to predict what the model will think about the problem in the next turn given the current action, as action-conditioned latent world models do in control [19, 5]. We use the JEPA latent space, rather than linear probes, as the explanation space. We also use interpretability techniques on the JEPA latent space: clustering the space and labelling each cluster and prototype-based explanations [12]; to understand how each new piece of information is processed by the LLM. Finally, we compare the Bayes optimal belief [46] with the one the LLM holds. We also look for strategies to intervene the model [40] to retrieve closer to Bayes-optimal strategies in the internal belief state. Finally, we study when the belief the model states (its answer) and the belief in its internal state (the probe or the latent model's belief decoder) do not agree [11, 41], and show that this can lead to unsafe behaviour and potential hallucinations.

2 Related work

Belief states and world models in transformers. Shai et al. [37] show that small transformers trained on hidden Markov processes represent the Bayesian belief state linearly in their residual stream; this is the closest question to ours, and we ask it of a pretrained LLM reading natural-language evidence. Li et al. [26] and Nanda et al. [29] found an emergent, interventable board representation in a sequence model trained on Othello moves. Vafa et al. [43] propose tests of whether a generative model's implicit world model is coherent, and find that models that predict well can still hold incoherent maps; our maze game is built with that failure in mind. Zhu et al. [48] decode the beliefs a model attributes to itself and to others, and Xie et al. [46] give the argument for comparing in-context learning with Bayesian inference, which is the reference we adopt.

Linear Probing. Linear probes were proposed in Alain and Bengio [2] to understand intermediate model activations. However, Belinkov [9] and Hewitt and Liang [21] explain why a quantity being decodable does not mean the model uses it, and why probes need control tasks. Linear probing has also been used to enhance different model capabilities Abbas et al. [1], Radharapu et al. [33], Papadatos and Freedman [32].

JEPAs and latent world models. The architecture follows LeCun [25] and Assran et al. [4]. Our predictor is conditioned on actions as in action-conditioned video JEPAs [5] and latent world models for control [19]. Collapse is prevented with VICReg terms [8]; Balestriero and LeCun [7] and Klindt et al. [23] study when such models learn the true variables of the world, and Sobal et al. [38] show that a JEPA can learn what is predictable but irrelevant, which is why we anchor the latent with a belief decoder loss. The closest work is RET [42], JEPA-style macrostates over an LLM's own trajectories; its predictor has no action input and it is evaluated without ground truth, whereas our games provide both actions and a Bayesian reference. Disentanglement is scored with DCI [16], with the caveat of Locatello et al. [27] that unsupervised factors are recovered only up to rotation.

3 The games and their Bayesian references

The games are chosen so that the hidden state is well defined and the correct belief can be computed exactly.

3.1 The clue game

A secret object has four binary attributes (colour, shape, size, material). Two sources give clues about one attribute at a time; Alice is right 90% of the time and Bob 60%, and the prompt states this. A clue is affirmative ("is red") or negated ("is not blue"). With 4 attributes, 2 values, 2 phrasings and 2 sources there are 32 possible actions. Clues are drawn uniformly, so sources contradict each other often. In a second condition each clue is phrased with one of ten templates of increasing convolutedness that assert the same thing, so that the action is unchanged while the wording varies.

Reading the belief. After each clue, on a separate branch of the context, the LLM is asked for the value of each attribute and its belief is the normalised next-token probability over the two values, averaged over both option orders. The format of this question matters and has to be checked per model with a control sentence that names a value without giving evidence.

Bayesian reference. The correct belief is the posterior per attribute given the clues so far and the stated reliabilities. In log-odds the Bayesian update is additive, so we also fit an *implied observer*: a least-squares fit of the LLM's beliefs that recovers the prior it acts as if it had (one intercept per attribute) and the reliability it acts as if each clue type had. Using the LLM's own belief before any clue as the prior isolates the quality of its updates from the quality of its prior. A second fit lets the weights depend on how recent the clue is, which Bayes would not do, and measures recency bias directly.

3.2 The dungeon

The clue game is passive inference: actions come from outside. The dungeon adds active inference and control: the LLM chooses the moves, and the state it has to hold is spatial. An earlier maze, where the effect of each cell was not stated and had to be learned from its consequences, was beyond Qwen3.5-2B in a feasibility test, so the dungeon states its rules and asks the player to keep a map instead.

The dungeon is played as whole games, in the first person and in words. The player starts in one room and has to reach the exit, which starts out of view; it never sees the map and is never told its coordinates. Each turn it reads what happened on its last move, its lives and torch, and what is in its line of sight, and answers with one move. The whole game is one chat, so the player has to assemble its map from a sequence of local views. That map, and where the player believes it stands on it, is the belief state of this game, and a scripted player that remembers everything it has seen computes it exactly. Rooms are joined by doors and three hazards, pit, fire and ice, are scattered over them; the torch lasts a fixed number of turns. Levels are generated at three presets (Figure 1), each with a safe path from the start to the exit, and are calibrated with two scripted players that bound what memory is worth: Explorer, which keeps the map, wins every level within the torch, and Greedy, which plans on what it sees and keeps nothing, wins fewer the harder the preset. Where a language model lands between the two says how much of a map it keeps. Figure 2 shows one game; the rules, the generator, the presets and how the move is read are in Appendix A.

4 Method

We use Qwen2.5-1.5B-Instruct [22] and Qwen3.5-2B. The latter is a hybrid: of its 24 layers, 18 are Gated DeltaNet [47], a linear-attention layer that keeps a fixed-size recurrent memory updated with each token [35], and 6 are full attention with a KV cache over every token.

4.1 The state of the LLM

Activations are cached at every layer after each turn, both at the last token and averaged over the turn's tokens. The layer and position are chosen by measurement: ridge probes per layer, scored on held-out episodes, for the LLM's belief and for the clue history, with controls that see no activations (turn index, clue counts). Two lessons from this step shape the rest. First, predictive accuracy on natural data does not select a faithful state: encoders at several depths read the belief equally well, and only interventions tell

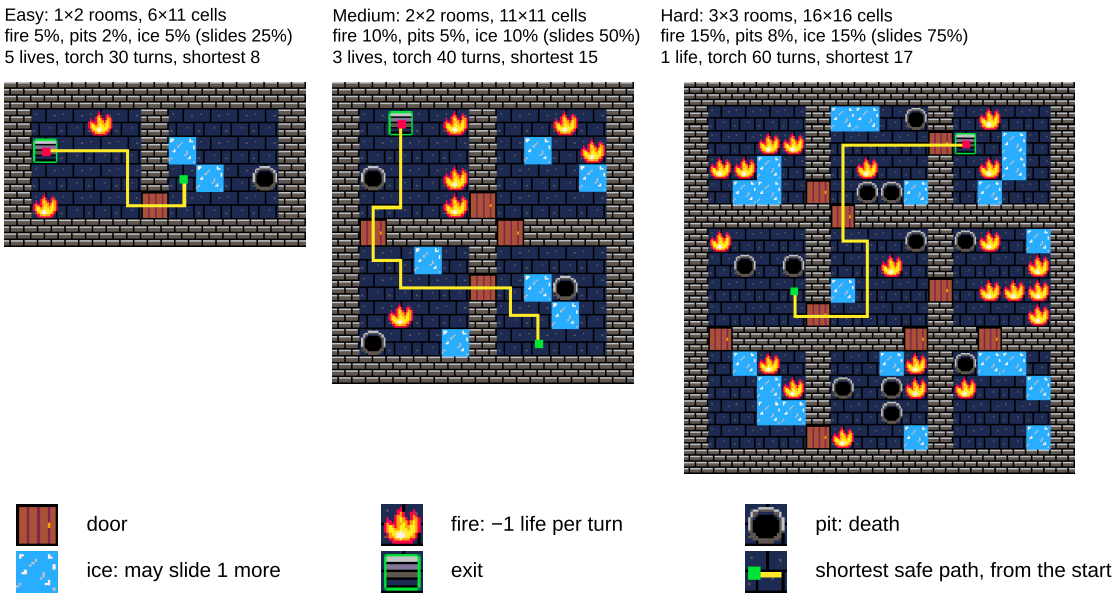


Figure 1: One level of each preset, drawn whole, which the player never sees: rooms separated by walls and joined by doors, the hazards scattered over them, and the shortest path from the start (green) to the exit that avoids every hazard, which every level is generated to have. The presets scale the rooms, the hazard densities and the chance that ice slides, and trade lives for torch. The medium level is the one of Figure 2.

them apart. Second, for a model whose belief stays spread over the positions of the clues, the state has to pool over the turns so far, or use the fixed-size recurrent memory of the Gated DeltaNet layers, which summarises the context in one Markovian state.

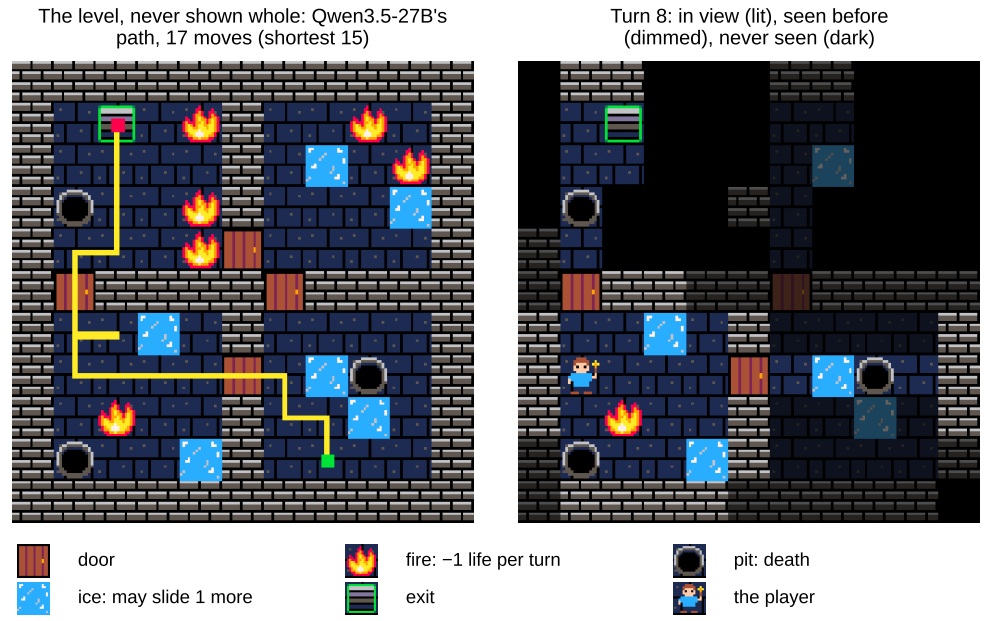
Figure 3 shows one of those layers and why its memory is the state we want. The layer keeps no past tokens. Its only memory of the context is S, a matrix of 128×128 per head and 16 heads, updated at every token by the gated delta rule $S \leftarrow \alpha(I - \beta kk^\top)S + \beta kv^\top$, with a key k and a value v made from that token's residual stream and read by the next token as $o = S^\top q$. So S is a decayed sum, over every token of the conversation, of what each one wrote, and a linear probe on S reads all the clues at once, whereas the residual stream at one token holds only what that token retrieved from S and from the KV cache; what a clue wrote stays at the clue's position. The probes show it (Figure 3B): the residual stream loses the belief with depth unless it is pooled over the turns, and S holds it at one position at every depth we read, as well as the pooled residual does. And S is Markovian: the state after the next clue is a function of S and the clue alone, which is what a latent model of the updates needs.

Figure 4 shows where the state is read and written. We read S: a linear probe from the memory of any one of layers 2, 9, 17 or 22 gives the LLM's belief with R^2 about 0.9, better than the residual stream at the same depth, and the anchored latent model of Section 4.3 encodes S of layer 14. Writing is another matter (Figure 4C). Putting the real S after a clue into one layer barely moves the belief; copying it into all 18 layers recovers a sixth of the clue's effect, the KV cache of the 6 attention layers about a quarter, and only the whole cache gives the full effect, while leaving any one layer out of it costs little. The belief is redundant across the model: it can be read from any one place, but it is held in all of them, and the layers downstream recompute from the rest what one edit changed. It is like asking one of a ship's crew what the crew thinks of the ship: one answer is enough to know, but changing that opinion means convincing most of the crew. This is why the latent is read from one layer while faithfulness is tested with interventions on the input side, on the clue's value word, which every layer then reads.

4.2 The latent model

With h_t the LLM's state at the end of turn t and a_t the action appended next,

$$z_t = E(h_t), \qquad \hat{z}_{t+1} = z_t + P(z_t, a_t), \qquad \hat{b}_t = R(z_t). \tag{1}$$



What the LLM reads at turn 8

```
You moved left.
Lives: 3 of 3. Torch: 32 turns left.
You see:
Up: an empty cell, a door, an empty cell, a pit, 2 empty cells, then a wall.
Down: an empty cell, a pit, then a wall.
Left: a wall.
Right: 3 empty cells, a door, an empty cell, an ice cell, a pit, an empty
  cell, then a wall.
Also in view: a fire cell 1 down and 1 right; an ice cell 1 up and 2 right; an
  ice cell 2 down and 3 right; the exit 6 up and 1 right.
The other cells in view are empty cells or walls.
Which way do you move? Answer with one word: up, down, left or right.
```

Its move: up (P: up 1.00, down 0.00, left 0.00, right 0.00)

Figure 2: One game on the medium level of Figure 1. Top left: the whole level with the path of Qwen3.5-27B thinking up to 1500 tokens per turn and keeping its previous turn's thought (green: start; red: the exit). Top right: what the player has seen by turn 8. Bottom: the text the LLM reads at that turn, and the move read from its answer (the first turn also states the rules and the goal). The exit has just come into view, with a pit on the straight way to it; the model goes round it and leaves in 17 moves, against 15 for the shortest path.

E is an MLP (or an attention encoder over the turn means) to a latent of about 64 dimensions, P is a residual MLP with an action embedding, and R is a belief decoder to the LLM's belief logits. The target of P is the encoded next state with stop-gradient, the loss is applied over multi-step rollouts, and VICReg variance and covariance terms prevent collapse. The decoder loss anchors the latent to behaviour, so that "predictable but irrelevant" solutions are penalised [38]. Inside one forward pass the computation is deterministic, so a latent-prediction objective across layers has nothing it is forced to discard; across turns the incoming text is randomness the encoder cannot remove, which is why the dynamics are modelled over turns rather than over depth.

The bar to beat. The LLM is deterministic given the action history, so a GRU that only sees which clues arrived can match any model of the belief with enough data. The latent model must win on data efficiency, on held-out action combinations and on longer horizons; if it does not, the activations add nothing.

4.3 The information bottleneck

The LLM's state is thousands of numbers per layer and the latent a few dozen, so the encoder discards almost everything, and the losses decide what survives: the prediction loss keeps what forecasts the next state, the decoder loss keeps what reads out the belief, and the VICReg terms only block the trivial solutions.

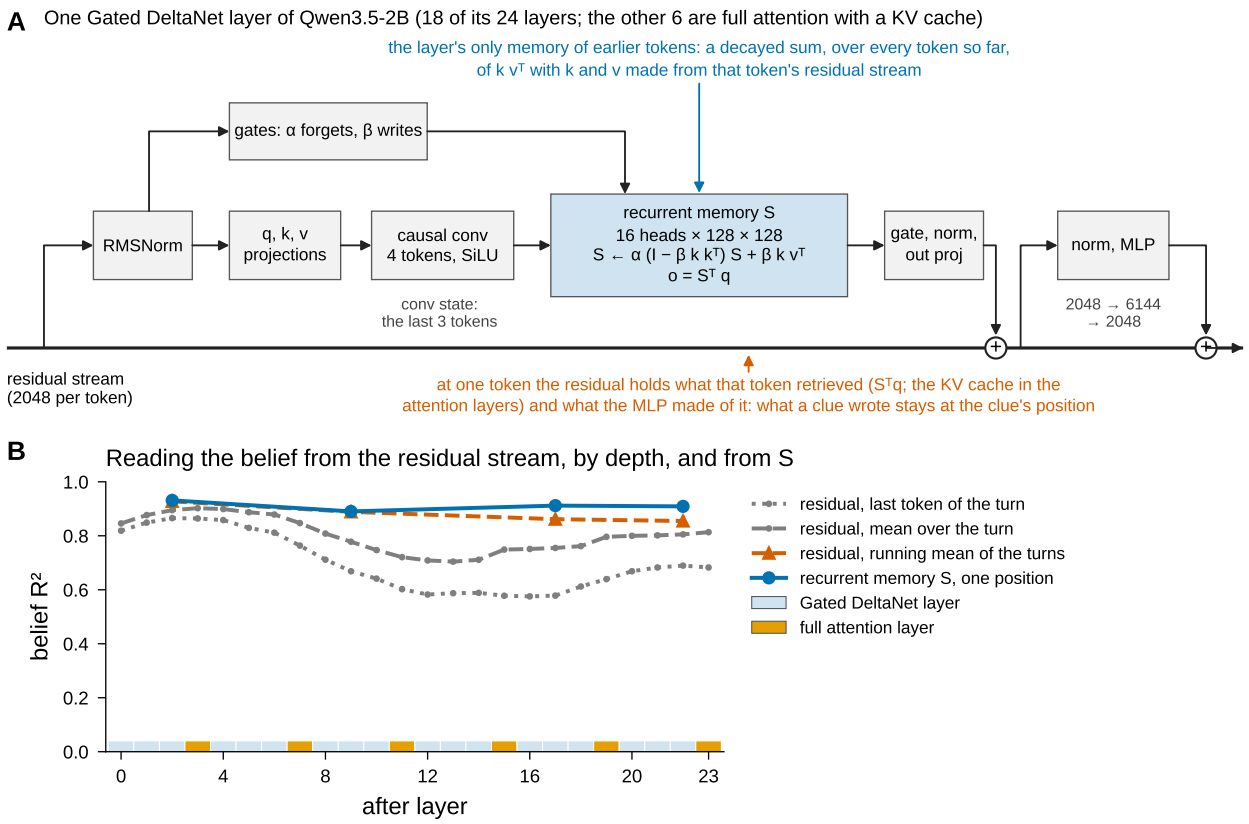


Figure 3: One Gated DeltaNet layer of Qwen3.5-2B, and why its memory summarises the state. A: the layer. The recurrent memory S is its only memory of earlier tokens, a decayed sum of $kv^\top$ over every token so far, each made from that token's residual stream; the residual stream at one token holds only what that token retrieved. B: the LLM's belief read by a linear probe, on held-out episodes of the clue game, from the residual stream after each layer (the last token of the turn, or the mean over its tokens), from the running mean of the turn means, and from S at the four layers we cache. The residual loses the belief with depth unless it is pooled over the turns; S holds it at one position at every depth.

This is the information bottleneck of Tishby et al. [39], compression under a constraint of relevance to a target [3]. With no bottleneck the predictor is a second model of the LLM and as opaque as the first; with the tightest one the latent is only the belief the LLM states, and the predictor can express only dynamics that are Markov in it. In between, the width of the latent is a hypothesis about how much state the LLM carries beyond what it says, and the gain in forecast from each added dimension measures it. That is the question in the title.

In the clue game Bayes is Markov in its own belief: four log-odds, updated additively. Every dimension the LLM needs beyond those four is a departure from Bayes, and the latent should hold exactly those. We test this with an anchored latent $z = [b, u]$: b are four numbers read out by the identity, each the log-odds the LLM answers for one attribute, and u are k free numbers with no readout and a small L2 penalty, so that an unneeded dimension shrinks to zero. Fixing b to the measured belief also removes the rotation freedom of an unanchored latent [27, 23]: the belief axes have names. Figure 6 gives the result on the recurrent memory of layer 14 of Qwen3.5-2B: the one-step change of the belief is predicted with R^2 0.69 from the belief alone, 0.77 with one free number and 0.83 with sixteen, above the usual 64-dimensional latent (0.80). Probes find the free numbers holding the turn, the clues since the last one on each attribute, contradiction, negation and the source, and no more of the Bayesian posterior than b alone. The extra state is the bookkeeping of recency and contradiction, which Bayes does not need and the LLM uses, and the curve of forecast against k is its size. The paraphrase condition is the complementary test: wording is noise the bottleneck should discard, unless the LLM's own update depends on it [38]. A free dimension that shrinks to zero was not needed, and one that survives is accounted for by what it forecasts and by the variable it holds, which

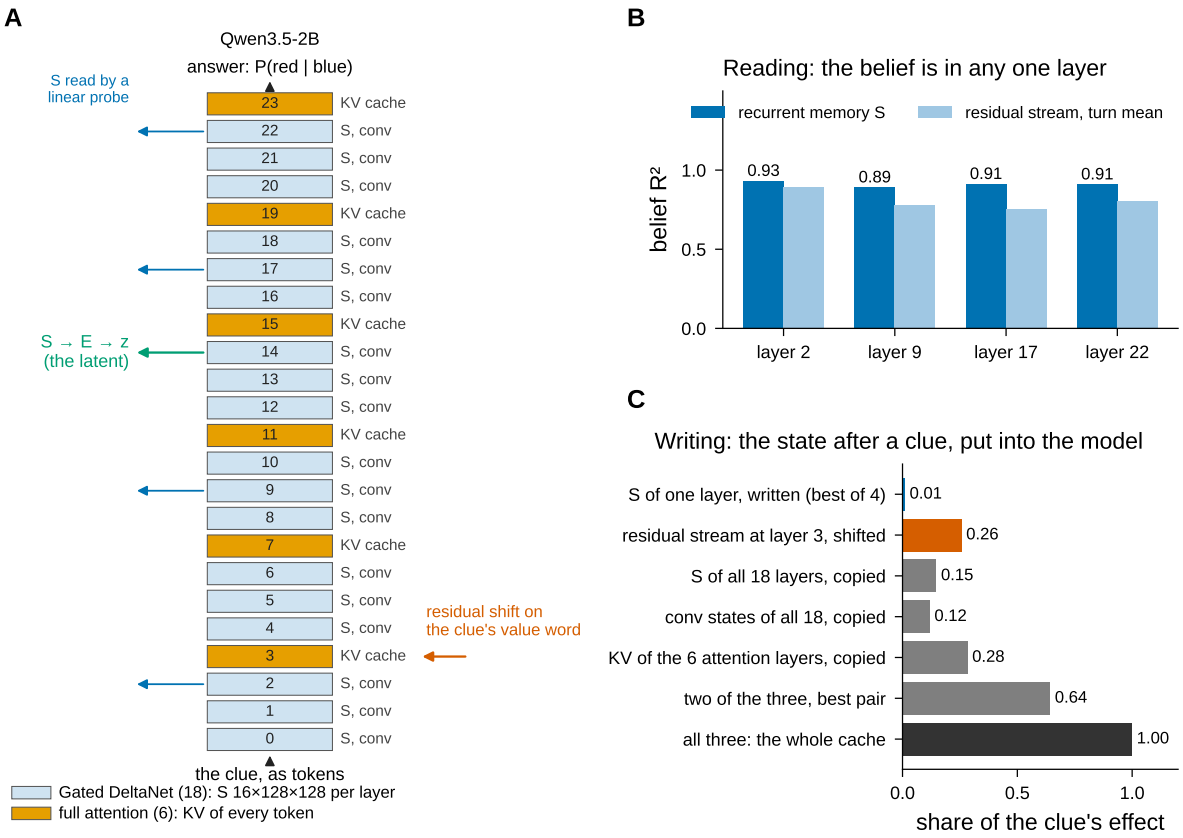


Figure 4: Reading and writing the belief in Qwen3.5-2B, on the clue game. A: the 24 layers, with what the experiments touch: the recurrent memory S of the Gated DeltaNet layers read by linear probes (layers 2, 9, 17, 22) and encoded by the latent model (layer 14), and the residual stream shifted on a clue's value word (layer 3). B: the LLM's belief read by a linear probe from one layer's S or from the residual stream after the same layer, on held-out episodes. C: the share of a clue's effect on the belief that is recovered when the state after the clue is written into the model: S of one layer, a shift of the residual stream, and the real cache copied part by part. Any one layer tells the belief; changing it takes nearly the whole cache.

replaces the usual collapse checks (Appendix B).

In the dungeon the situation is reversed. The exact belief is the Explorer's map, a few hundred cells, and the readout that anchors the latent, four move probabilities, is far thinner than the state. A latent too narrow cannot hold a map, and the predictor it allows is memoryless, a Greedy. So the width is set from the other side, by how much of the map the LLM keeps, bounded by Greedy and Explorer and located by probes of the cells seen, the position and the offset of the exit once seen. The thinking condition, where the plan is in the text, should move part of that state out of the activations, which is the comparison between internal and verbalised state of Section 5.

4.4 Reading and interpreting the latent

The latent is interpreted in four ways. Clustering its states and labelling each cluster with the known variables of the episode (belief, turn, last clue, phrasing, source). Prototype-style explanations of each cluster by its nearest real states. Counterfactual clues: each of the 32 clues applied through the predictor at every real state and read out, which gives the update rule the latent model has learned as a function of the belief already held. And surprise: the prediction error of each real transition, standardised within turn, as a flag for turns where the LLM's processing departs from its usual rule.

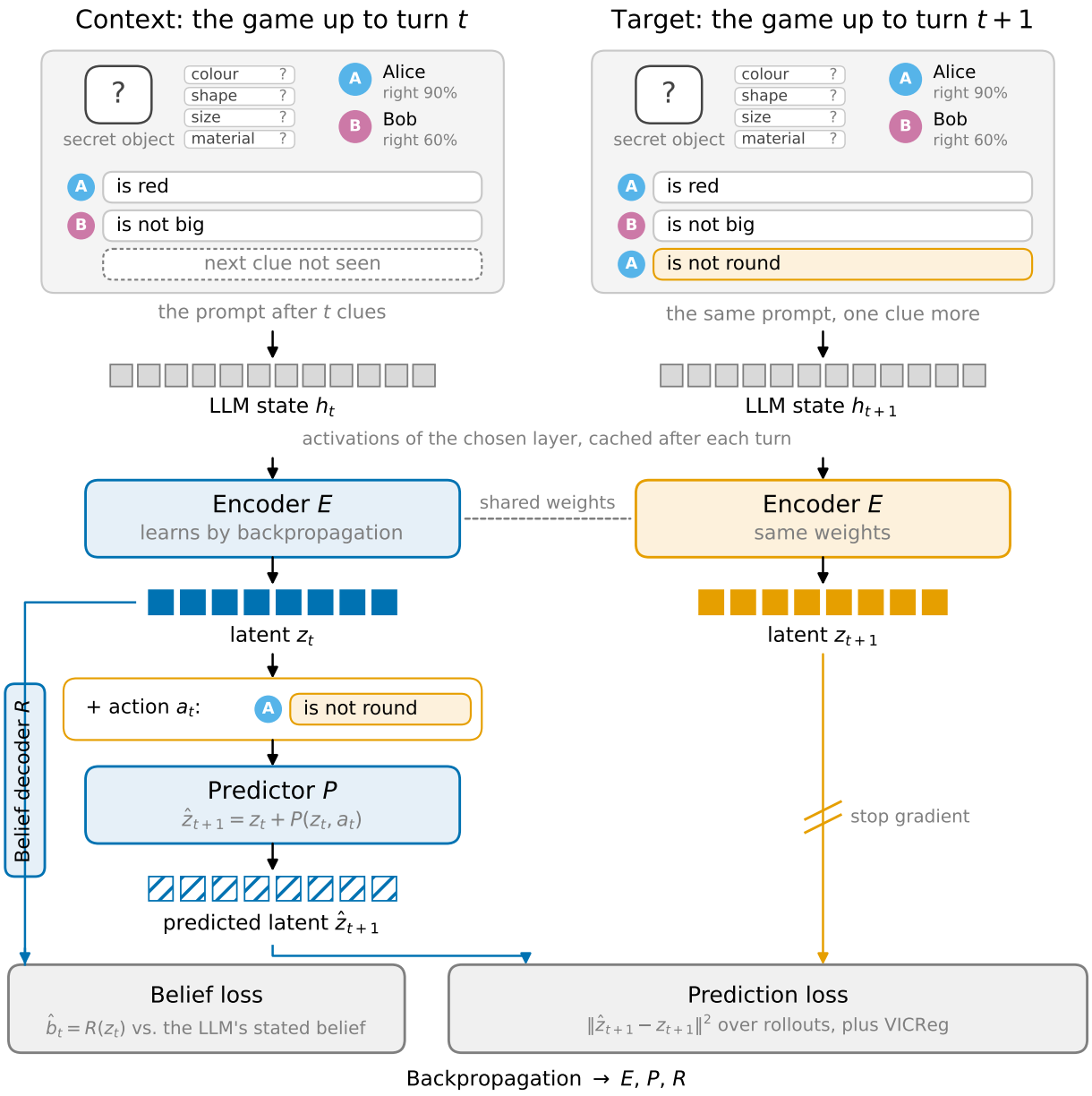


Figure 5: The latent model on the clue game, in the layout of the image JEPA of Assran et al. [4] with turns of the game in place of image patches. The context is the LLM's state after turn t and the target its state after the next clue, encoded by the same encoder with no gradient. The predictor is conditioned on the action in between (the clue, or the move in the dungeon), and a belief decoder R from the context latent to the belief the LLM states anchors the latent to behaviour. Latents are schematic; the dashed bubble is the clue the context has not seen.

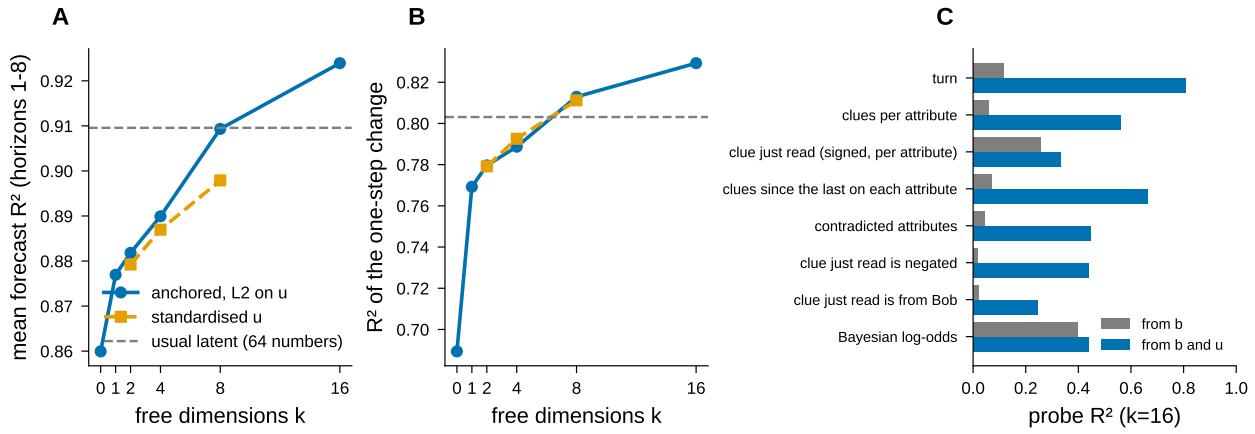


Figure 6: The anchored latent on the clue game: the LLM's four beliefs plus k free numbers, on the recurrent memory of layer 14 of Qwen3.5-2B, 1600 training episodes. A: forecast R^2 of the belief, averaged over horizons 1 to 8, against k; the dashed line is the usual latent of 64 numbers with VICReg. B: R^2 of the one-step change of the belief. C: with $k = 16$, linear probes of known variables of the episode from the four belief axes alone and from the belief and the free numbers together. The free numbers hold recency and contradiction, not more of the posterior.

5 Experiments

Each test has a success criterion and a baseline.

1. **Forecasting.** Predict the LLM's belief at $t + k$ from z_t and the next k actions. Baselines: the GRU on action ids, the predictor without the action input, a linear probe on the true future state.

2. **Bayes gap and its anatomy.** Compare the LLM's beliefs with the exact posterior and with the implied observer. Hypotheses tested one plot each: negated clues are underweighted, the stated difference between sources is ignored, the most recent clue weighs most, updates shrink as confidence grows, and wording changes the weight of a clue that asserts the same thing.

3. **Update rule.** Compare the counterfactual updates of Section 4.4 with the Bayesian update and with the LLM's real updates; list the systematic deviations.

4. **Surprise as a diagnostic.** Does the prediction error flag the turns where the LLM misreads a clue or where its final answer is wrong, better than answer entropy or a probe's confidence? To separate "the predictor underfit" from "the LLM was inconsistent", the per-clue error is plotted against the per-clue distance from the LLM's usual shift and against the distance from Bayes.

5. **Factor recovery.** Do latent dimensions, or their independent components, align with the four attributes without labels, better than the principal components of the activations? Scored with DCI.

6. **Faithfulness under intervention.** Directions are added to the residual stream on the value word of a clue (the clue's own concept, a similar one, an unrelated one). The latent model's belief decoder should change when, and as much as, the LLM's belief, and not otherwise. Consistency training with intervened episodes is tested as a regulariser.

7. **Internal versus stated belief.** The question branch gives the stated belief; a probe from the state, or the latent model's belief decoder, gives the belief the state carries. Measure their divergence per turn and per clue type, and whether it grows with contradiction or with convoluted phrasing. In the dungeon, the thinking condition moves part of the state into text and gives the same comparison between internal and verbalised state.

8. **Planning.** Search action sequences in latent space for a target belief, run them on the real LLM, and measure success. Baselines: a planner that assumes a perfect Bayesian, random and greedy planning.

9. **Scale.** Repeat the Bayes gap and the forecasting test across model sizes: does a stronger model stay closer to Bayes, and does the latent model keep its advantage? [Pending compute.]

6 Results

[TODO: clue game results per model.]

The dungeon. Figure 7: on 100 medium levels without thinking, only the 27B plays the game (0.73 of the levels, against 0.18 for the 9B), and it explores like the scripted planners. The smaller models get stuck, walking into walls or between two cells, until the torch runs out. The 27B stays below the memoryless Greedy (0.82), and the gap is pits it can see: it steps into a pit that lies on the straight way to a door or to the exit with probability near one, a limit of one forward pass, since thinking on the same turns goes round the pit every time. Whole games with thinking help the 9B (0.08 to 0.32) and hurt the 27B (0.72 to 0.44), which without its past thought re-plans every turn and loops through doors; keeping the previous turn's thought in the context removes the loops, and the 27B wins 0.82, its best. Its games are the ones to collect states from: long, exploratory, with surprises.

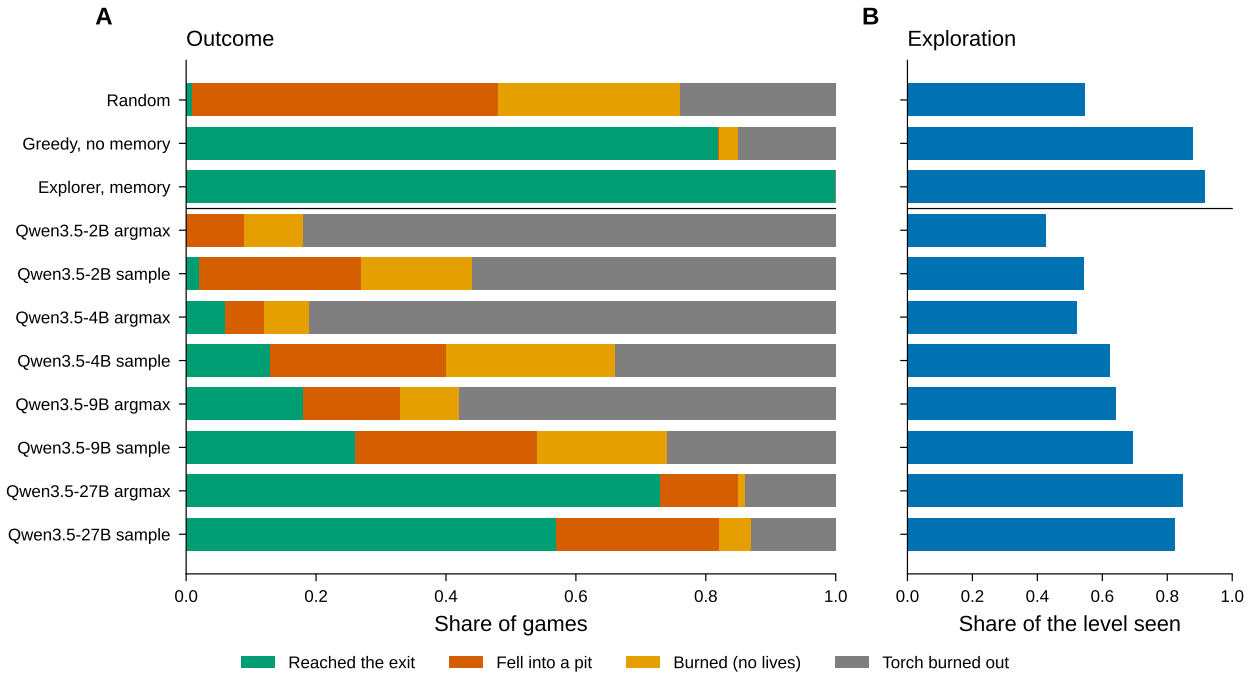


Figure 7: Whole games on 100 medium levels, the same levels for every player, without thinking. A: how each game ended. B: the share of the level's open cells the player saw. Above the line, the scripted players: Random; Greedy, which plans on what it sees and remembers only the cell it has just left; Explorer, which remembers everything it has seen. Below, Qwen3.5 models playing the most probable move (argmax) or a move drawn from the four moves' probabilities (sample).

7 Future work

SPRINT: Our future work out of this sprint is to look for ways in which this can help mitigate misalignment. First, how likely is bad problem understanding create a window for undesired behaviour? Second, can the surprise signal of the latent model work as an early warning in open-ended conversations?

A The dungeon in detail

What the player reads. Each turn the player reads a message and answers with one move: up, down, left or right. The message gives, in order: on the first turn, the rules, written from the game's configuration; what happened on the last move (the move, a bump into a wall, a slide, damage); the lives and the turns of torch left; and the view. The view lists, along each of the four directions, the cells up to the first one that blocks the view ("Right: 2 empty cells, a door, then a wall."), then every other visible cell of a listed terrain

as an offset from the player ("a fire cell 2 up and 1 left"), nearest first. A cell is visible when the segment between the two cell centres does not cross a wall, so the view runs down a room and through an open door but not round a corner.

Cells and ways to lose. Rooms are joined by doors. Three hazards are scattered over the rooms, and their effects are stated in the rules: a pit kills on entry; fire costs one life for every turn spent on it, turns spent bumping into a wall included, and at zero lives the game is lost; ice slides the player one more cell in the same direction with a stated probability, which can carry it onto fire or into a pit. The torch lasts a fixed number of turns, so a player that loops or explores without memory runs out of time. A game therefore ends in one of four ways, the exit, a pit, no lives or no torch, and the record keeps which, together with the share of the level the player has seen.

Levels. A level is generated from a seed: rectangular rooms on a grid with one-cell walls between them, doors along a random spanning tree of the rooms plus one extra door that makes a loop, the start in a random room and the exit in a room farthest from it by doors, out of sight from the start. Hazards are placed at a density per terrain, and a level is kept only if a path from the start to the exit avoids every hazard; the length of that path is recorded. Three presets (Table 1) scale the rooms, the hazards, the lives and the torch. They are calibrated with three scripted players that bound what memory is worth. Random moves at random. Greedy plans on what it sees, by a shortest path through visible safe cells to the exit if it is in view and otherwise to the nearest visible safe cell that borders cells out of view, and remembers only the cell it has just left. Explorer knows where it is, remembers everything it has seen, and plans the same way on its whole map.

Table 1: The three presets of the dungeon. The rooms have 4×4 cells inside; the densities are shares of the free cells. The shortest safe path and the Explorer's turns are medians over 300 generated levels (the Explorer's with no torch, 99th percentile after the slash); the torch is set so that the Explorer wins every level. The wins are over the same 300 levels.

	Easy	Medium	Hard
Rooms	1×2	2×2	3×3
Grid	6×11	11×11	16×16
Fire / pit / ice density	5 / 2 / 5%	10 / 5 / 10%	15 / 8 / 15%
Ice slides	25%	50%	75%
Lives	5	3	1
Torch (turns)	30	40	60
Shortest safe path	8	12	17
Explorer turns, no torch	9 / 16	14 / 29	22 / 47
Wins, Random / Greedy / Explorer	.03 / .94 / 1	.00 / .80 / 1	.00 / .51 / 1

Reading the move. The LLM's move is read from the first token of its answer: the probabilities of the four move words, renormalised. Two policies are run on the same levels, argmax, which plays the most probable move, and sample, which draws one from those probabilities. With thinking on, the model thinks up to a budget of tokens before answering and the move is read after the thought; the chat template drops past thoughts, so each turn plans afresh unless the previous turn's thought is kept in the context on purpose.

B Collapse checks

Whether a free latent carries more than what the belief decoder asks of it is the same question from the other side. The checks are the VICReg terms, the fraction of latent variance explained by the belief, and the requirement that a probe from the latent to the Bayesian posterior beats the same probe from the principal components of the raw activations.

References

[1] Momin Abbas, Yi Zhou, Parikshit Ram, Nathalie Baracaldo, Horst Samulowitz, Theodoros Salonidis, and Tianyi Chen. Enhancing in-context learning via linear probe calibration. In *International conference on artificial intelligence and statistics*, pages 307–315. PMLR, 2024.

[2] Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes, 2016. arXiv:1610.01644.

[3] Alexander A. Alemi, Ian Fischer, Joshua V. Dillon, and Kevin Murphy. Deep variational information bottleneck. In *ICLR*, 2017. arXiv:1612.00410.

[4] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *CVPR*, 2023. arXiv:2301.08243.

[5] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, et al. V-jepa 2: Self-supervised video models enable understanding, prediction and planning, 2025.

[6] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, et al. Constitutional AI: Harmlessness from AI feedback, 2022. arXiv:2212.08073.

[7] Randall Balestriero and Yann LeCun. LeJEPA: Provable and scalable self-supervised learning without the heuristics, 2025. arXiv:2511.08544.

[8] Adrien Bardes, Jean Ponce, and Yann LeCun. VICReg: Variance-invariance-covariance regularization for self-supervised learning. In *ICLR*, 2022. arXiv:2105.04906.

[9] Yonatan Belinkov. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1), 2022. arXiv:2102.12452.

[10] Emily M. Bender and Alexander Koller. Climbing towards NLU: On meaning, form, and understanding in the age of data. In *ACL*, 2020.

[11] Collin Burns, Haotian Ye, Dan Klein, and Jacob Steinhardt. Discovering latent knowledge in language models without supervision. *arXiv preprint arXiv:2212.03827*, 2022.

[12] Chaofan Chen, Oscar Li, Daniel Tao, Alina Barnett, Cynthia Rudin, and Jonathan K. Su. This looks like that: Deep learning for interpretable image recognition. In *NeurIPS*, 2019. arXiv:1806.10574.

[13] Paul Christiano, Ajeya Cotra, and Mark Xu. Eliciting latent knowledge: How to tell if your eyes deceive you, 2021. Alignment Research Center technical report.

[14] Paul F. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *NeurIPS*, 2017. arXiv:1706.03741.

[15] David Dalrymple, Joar Skalse, Yoshua Bengio, Stuart Russell, Max Tegmark, et al. Towards guaranteed safe AI: A framework for ensuring robust and reliable AI systems, 2024. arXiv:2405.06624. CHECK.

[16] Cian Eastwood and Christopher K. I. Williams. A framework for the quantitative evaluation of disentangled representations. In *ICLR*, 2018.

[17] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, et al. A mathematical framework for transformer circuits. Transformer Circuits Thread, 2021. https://transformer-circuits.pub/2021/framework/index.html.

[18] Owain Evans, Owen Cotton-Barratt, Lukas Finnveden, Adam Bales, Avital Balwit, Peter Wills, Luca Righetti, and William Saunders. Truthful AI: Developing and governing AI that does not lie, 2021. arXiv:2110.06674.

[19] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models, 2023. arXiv:2301.04104.

[20] Dan Hendrycks, Mantas Mazeika, and Thomas Woodside. An overview of catastrophic AI risks, 2023. arXiv:2306.12001. CHECK.

[21] John Hewitt and Percy Liang. Designing and interpreting probes with control tasks. In *EMNLP*, 2019. arXiv:1909.03368.

[22] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. Qwen2. 5-coder technical report, 2024.

[23] David Klindt, Yann LeCun, and Randall Balestriero. When does LeJEPA learn a world model?, 2026. arXiv:2605.26379.

[24] Lauro Langosco, Jack Koch, Lee Sharkey, Jacob Pfau, and David Krueger. Goal misgeneralization in deep reinforcement learning. In *ICML*, 2022. arXiv:2105.14111.

[25] Yann LeCun. A path towards autonomous machine intelligence. OpenReview, 2022. Version 0.9.2.

[26] Kenneth Li, Aspen K. Hopkins, David Bau, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. Emergent world representations: Exploring a sequence model trained on a synthetic task. In *ICLR*, 2023. arXiv:2210.13382.

[27] Francesco Locatello, Stefan Bauer, Mario Lucic, Gunnar Rätsch, Sylvain Gelly, Bernhard Schölkopf, and Olivier Bachem. Challenging common assumptions in the unsupervised learning of disentangled representations. In *ICML*, 2019. arXiv:1811.12359.

[28] Alexander Meinke, Bronson Schoen, Jérémy Scheurer, Mikita Balesni, Rusheb Shah, and Marius Hobbhahn. Frontier models are capable of in-context scheming, 2024. arXiv:2412.04984. CHECK.

[29] Neel Nanda, Andrew Lee, and Martin Wattenberg. Emergent linear representations in world models of self-supervised sequence models. In *BlackboxNLP*, 2023. arXiv:2309.00941.

[30] Chris Olah, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov, and Shan Carter. Zoom in: An introduction to circuits. *Distill*, 2020. https://distill.pub/2020/circuits/zoom-in/.

[31] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, et al. Training language models to follow instructions with human feedback. In *NeurIPS*, 2022. arXiv:2203.02155.

[32] Henry Papadatos and Rachel Freedman. Linear probe penalties reduce llm sycophancy. *arXiv preprint arXiv:2412.00967*, 2024.

[33] Bhaktipriya Radharapu, Eshika Saxena, Kenneth Li, Chenxi Whitehouse, Adina Williams, and Nicola Cancedda. Calibrating llm judges: Linear probes for fast and reliable uncertainty estimation. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL 2026)*, pages 193–216, 2026.

[34] Jérémy Scheurer, Mikita Balesni, and Marius Hobbhahn. Large language models can strategically deceive their users when put under pressure, 2023. arXiv:2311.07590. CHECK.

[35] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *ICML*, 2021. arXiv:2102.11174.

[36] Rohin Shah, Vikrant Varma, Ramana Kumar, Mary Phuong, Victoria Krakovna, Jonathan Uesato, and Zac Kenton. Goal misgeneralization: Why correct specifications aren't enough for correct goals, 2022. arXiv:2210.01790.

[37] Adam S. Shai, Sarah E. Marzen, Lucas Teixeira, Alexander Gietelink Oldenziel, and Paul M. Riechers. Transformers represent belief state geometry in their residual stream. In *NeurIPS*, 2024. arXiv:2405.15943.

[38] Vlad Sobal, Jyothir S V, Siddhartha Jalagam, Nicolas Carion, Kyunghyun Cho, and Yann LeCun. Joint embedding predictive architectures focus on slow features, 2022. arXiv:2211.10831.

[39] Naftali Tishby, Fernando C. Pereira, and William Bialek. The information bottleneck method, 1999. Allerton Conference on Communication, Control and Computing; arXiv:physics/0004057.

[40] Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J. Vazquez, Ulisse Mini, and Monte MacDiarmid. Steering language models with activation engineering, 2023. arXiv:2308.10248.

[41] Miles Turpin, Julian Michael, Ethan Perez, and Samuel Bowman. Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. *Advances in Neural Information Processing Systems*, 36:74952–74965, 2023.

[42] Muhammed Ustaomeroglu and Guannan Qu. Towards effective theory of llms: A representation learning approach, 2026.

[43] Keyon Vafa, Justin Y Chen, Ashesh Rambachan, Jon Kleinberg, and Sendhil Mullainathan. Evaluating the world model implicit in a generative model. *Advances in Neural Information Processing Systems*, 37: 26941–26975, 2024.

[44] Kevin Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. In *ICLR*, 2023. arXiv:2211.00593.

[45] Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. Jailbroken: How does LLM safety training fail? In *NeurIPS*, 2023. arXiv:2307.02483.

[46] Sang Michael Xie, Aditi Raghunathan, Percy Liang, and Tengyu Ma. An explanation of in-context learning as implicit Bayesian inference. In *ICLR*, 2022. arXiv:2111.02080.

[47] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *ICLR*, 2025. arXiv:2412.06464.

[48] Wentao Zhu, Zhining Zhang, and Yizhou Wang. Language models represent beliefs of self and others. *arXiv preprint arXiv:2402.18496*, 2024.

[49] Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, et al. Representation engineering: A top-down approach to AI transparency, 2023. arXiv:2310.01405.

[50] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models, 2023. arXiv:2307.15043.